R4DS 04 – Tidy Data, Import & Workflow

Tidy Data

What is Tidy Data?

"Tidy datasets are all alike, but every messy dataset is messy in its own way." — *Hadley Wickham*

Three rules:

1. Each **variable** is a column
2. Each **observation** is a row
3. Each **value** is a cell

Why tidy? Consistent structure → tools just work (dplyr, ggplot2, etc.).

Tidy vs. Messy

```
table1
```

```
#> # A tibble: 6 x 4
#>   country      year  cases population
#>   <chr>       <dbl>  <dbl>      <dbl>
#> 1 Afghanistan  1999    745   19987071
#> 2 Afghanistan  2000   2666   20595360
#> 3 Brazil       1999  37737  172006362
#> 4 Brazil       2000  80488  174504898
#> 5 China        1999 212258 1272915272
#> ... [output truncated]
```

This is tidy: each row is a country-year, each column is a variable.

Messy Example: Values in Column Names

```
table2
```

```
#> # A tibble: 12 x 4
#>    country      year type            count
#>    <chr>       <dbl> <chr>           <dbl>
#>  1 Afghanistan  1999 cases             745
#>  2 Afghanistan  1999 population   19987071
#>  3 Afghanistan  2000 cases            2666
#> ... [output truncated]
```

Here `type` mixes two variables (`cases` and `population`) into one column.

```
# Rate per 10,000
table1 |>
  mutate(rate = cases / population * 10000)
```

```
#> # A tibble: 6 x 5
#>   country      year  cases population  rate
#>   <chr>       <dbl>  <dbl>      <dbl> <dbl>
#> 1 Afghanistan  1999    745   19987071 0.373
#> 2 Afghanistan  2000   2666   20595360 1.29
#> ... [output truncated]
```

```
# Total cases per year
table1 |>
  group_by(year) |>
  summarize(total_cases = sum(cases))
```

```
#> # A tibble: 2 x 2
#>    year total_cases
#>   <dbl>       <dbl>
#> 1  1999      250740
#> ... [output truncated]
```

Exercises (Tidy Data)

1. For each sample table (`table1`, `table2`, `table3`), describe what each observation and column represents.
2. Sketch the process to calculate `rate` for `table2` and `table3`: (a) extract cases per country-year, (b) extract population, (c) divide and multiply by 10,000, (d) store result.

Solutions (Tidy Data)

1. `table1`: each row = country-year; columns = country, year, cases, population. `table2`: each row = country-year-type; `count` mixes cases and population. `table3`: each row = country-year; `rate` column stores "cases/population" as a string.
2. For `table2`: filter rows where `type == "cases"`, filter where `type == "population"`, join, divide. For `table3`: use `separate()` to split the `rate` column by "/", convert to numbers, divide.

Pivoting

`pivot_longer()` — Column Names to Values

The `billboard` dataset has weeks as columns:

```
billboard
```

```
#> # A tibble: 317 x 79
#>    artist    track date.entered   wk1   wk2   wk3   wk4
#>    <chr>     <chr> <date>       <dbl> <dbl> <dbl> <dbl>
#>  1 2 Pac     Baby~ 2000-02-26      87    82    72    77
#>  2 2Ge+her   The ~ 2000-09-02      91    87    92    NA
#> ... [output truncated]
```

Pivoting Billboard Data

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank",
    values_drop_na = TRUE
  )
```

```
#> # A tibble: 5,307 x 5
#>    artist  track                   date.entered week   rank
#>    <chr>   <chr>                   <date>       <chr> <dbl>
#>  1 2 Pac   Baby Don't Cry (Ke~ 2000-02-26   wk1      87
#>  2 2 Pac   Baby Don't Cry (Ke~ 2000-02-26   wk2      82
#>  3 2 Pac   Baby Don't Cry (Ke~ 2000-02-26   wk3      72
#>  4 2 Pac   Baby Don't Cry (Ke~ 2000-02-26   wk4      77
#> ... [output truncated]
```

- `cols`: which columns to pivot
- `names_to`: new variable from old column names
- `values_to`: new variable from cell values

Cleaning After Pivot

```
billboard_longer <- billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week", values_to = "rank",
    values_drop_na = TRUE
  ) |>
  mutate(week = parse_number(week))
billboard_longer
```

```
#> # A tibble: 5,307 x 5
#>    artist  track                date.entered  week  rank
#>    <chr>   <chr>                <date>       <dbl> <dbl>
#>  1 2 Pac   Baby Don't Cry (Ke~ 2000-02-26       1    87
#>  2 2 Pac   Baby Don't Cry (Ke~ 2000-02-26       2    82
#>  3 2 Pac   Baby Don't Cry (Ke~ 2000-02-26       3    72
#> ... [output truncated]
```

Multiple Variables in Column Names

```
who2
```

```
#> # A tibble: 7,240 x 58
#>    country  year sp_m_014 sp_m_1524 sp_m_2534 sp_m_3544
#>    <chr>   <dbl>    <dbl>     <dbl>     <dbl>     <dbl>
#>  1 Afghan~  1980       NA        NA        NA        NA
#>  2 Afghan~  1981       NA        NA        NA        NA
#> ... [output truncated]
```

Columns like `sp_m_014`: diagnosis / gender / age range.

Splitting Column Names

```
who2 |>
  pivot_longer(
    cols = !(country:year),
    names_to = c("diagnosis", "gender", "age"),
    names_sep = "_",
    values_to = "count"
  )
```

```
#> # A tibble: 405,440 x 6
#>    country      year diagnosis gender age   count
#>    <chr>       <dbl> <chr>     <chr>  <chr> <dbl>
#>  1 Afghanistan  1980 sp        m      014      NA
#>  2 Afghanistan  1980 sp        m      1524     NA
#>  3 Afghanistan  1980 sp        m      2534     NA
#> ... [output truncated]
```

`names_sep` splits each column name into multiple variables.

pivot_wider() — Values to Column Names

```
cms_patient_experience |>
  pivot_wider(
    id_cols = starts_with("org"),
    names_from = measure_cd,
    values_from = prf_rate
  )
```

```
#> # A tibble: 95 x 8
#>    org_pac_id org_nm            CAHPS_GRP_1 CAHPS_GRP_2
#>    <chr>      <chr>                   <dbl>       <dbl>
#>  1 0446157747 USC CARE MEDICAL~          63          87
#>  2 0446162697 ASSOCIATION OF U~          59          85
#> ... [output truncated]
```

`pivot_wider()` is the inverse of `pivot_longer()`: spreads rows into columns.

Importing Data

`read_csv()` — Reading CSV Files

```
students <- read_csv("data/students.csv")
```

- First argument: path to file (or URL)
- Prints column names and types
- Returns a tibble

Practical Advice

```r
# Handle non-standard NA values
students <- read_csv("data/students.csv",
                     na = c("N/A", ""))

# Fix non-syntactic names
students |>
  rename(student_id = `Student ID`,
         full_name = `Full Name`)

# Or use janitor::clean_names() for all at once
students |> janitor::clean_names()
```

Other Useful Arguments

```
# Skip metadata lines
read_csv("metadata line 1
x,y,z
1,2,3", skip = 1)
```

```
#> # A tibble: 1 x 3
#>       x     y     z
#>   <dbl> <dbl> <dbl>
#> 1     1     2     3
```

```
# No header row
read_csv("1,2,3
4,5,6", col_names = c("x", "y", "z"))
```

```
#> # A tibble: 2 x 3
#>       x     y     z
#>   <dbl> <dbl> <dbl>
#> 1     1     2     3
#> 2     4     5     6
```

Other File Types

Function	Delimiter
`read_csv()`	Comma ,
`read_csv2()`	Semicolon ;
`read_tsv()`	Tab
`read_delim()`	Any (auto-detect)
`read_fwf()`	Fixed width

Column Types

readr guesses types: logical → number → date → string.
Override with `col_types`:

```
read_csv("file.csv",
         col_types = list(x = col_double()))
```

Fix mismatched types with `na` argument:

```
read_csv("x\n10\n.\n20", na = ".")
```

```
#> # A tibble: 3 x 1
#>       x
#>   <dbl>
#> 1    10
#> 2    NA
#> 3    20
```

Reading Multiple Files

```r
# Stack multiple CSVs into one data frame
sales_files <- list.files("data",
  pattern = "sales\\.csv$", full.names = TRUE)
read_csv(sales_files, id = "file")
```

`id = "file"` adds a column identifying the source file.

Writing Data

```r
# CSV (loses type info)
write_csv(students, "students.csv")

# RDS (preserves exact R object)
write_rds(students, "students.rds")
read_rds("students.rds")

# Parquet (fast, cross-language)
library(arrow)
write_parquet(students, "students.parquet")
```

Recommendation: use RDS for caching R objects, parquet for sharing.

Data Entry

```r
# By column
tibble(x = c(1, 2, 5), y = c("h", "m", "g"))
```

```
#> # A tibble: 3 x 2
#>       x y
#>   <dbl> <chr>
#> ... [output truncated]
```

```r
# By row (easier to read)
tribble(
  ~x, ~y,
   1, "h",
   2, "m",
   5, "g"
)
```

```
#> # A tibble: 3 x 2
#>       x y
#>   <dbl> <chr>
#> ... [output truncated]
```

Exercises (Import)

1. What function reads files delimited with `|`?
2. What arguments do `read_csv()` and `read_tsv()` share (besides `file`, `skip`, `comment`)?
3. Most important arguments to `read_fwf()`?
4. How do you read: `"x,y\n1,'a,b'"`?
5. What's wrong with each?

```
read_csv("a,b\n1,2,3\n4,5,6")
read_csv("a,b,c\n1,2\n1,2,3,4")
read_csv("a,b\n\"1")
read_csv("a,b\n1,2\na,b")
read_csv("a;b\n1;3")
```

6. Practice with non-syntactic names (extract, plot, mutate, rename).

Solutions (Import)

1. `read_delim(file, delim = "|")`
2. Nearly all: `col_names`, `col_types`, `locale`, `na`, `trim_ws`, `n_max`, `guess_max`, etc.
3. `file` and `col_positions` (or `fwf_widths()` / `fwf_positions()`).
4. `read_csv("x,y\n1,'a,b'", quote = "'")`
5. (a) 3 values but 2 columns — 3rd value dropped. (b) Mismatched columns per row. (c) Only one value, one column missing. (d) `a,b` in row 2 is character, forces column type to character. (e) Semicolons need `read_csv2()` or `read_delim()`.
6.

```
annoying <- tibble(`1` = 1:10, `2` = `1` * 2 + rnorm(10))
annoying$`1`                                    # extract
ggplot(annoying, aes(x = `1`, y = `2`)) + geom_point()
annoying |> mutate(`3` = `2` / `1`)
annoying |> rename(one = `1`, two = `2`)
```

Code Style

Naming Conventions

```
# Strive for:
short_flights <- flights |> filter(air_time < 60)

# Avoid:
SHORTFLIGHTS <- flights |> filter(air_time < 60)
```

- Use `snake_case` (lowercase + underscores)
- Prefer descriptive names over short ones
- Common prefix for related variables (autocomplete-friendly)

Spacing Rules

```r
# Strive for:
z <- (a + b)^2 / d
mean(x, na.rm = TRUE)

# Avoid:
z<-( a + b ) ^ 2/d
mean (x ,na.rm=TRUE)
```

Spaces around operators (+, −, ==, <−) and after commas. No spaces inside parentheses.

Pipe and ggplot2 Formatting

```r
# Strive for:
flights |>
  filter(!is.na(arr_delay), !is.na(tailnum)) |>
  group_by(dest) |>
  summarize(
    delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )

# ggplot2: treat + like |>
flights |>
  group_by(month) |>
  summarize(delay = mean(arr_delay, na.rm = TRUE)) |>
  ggplot(aes(x = month, y = delay)) +
  geom_point() +
  geom_line()
```

Exercises (Style)

Restyle the following:

```
flights|>filter(dest=="IAH")|>group_by(year,month,
day)|>summarize(n=n(),delay=mean(arr_delay,
na.rm=TRUE))|>filter(n>10)

flights|>filter(carrier=="UA",dest%in%c("IAH",
"HOU"),sched_dep_time>0900,sched_arr_time<2000)|>
group_by(flight)|>summarize(delay=mean(arr_delay,
na.rm=TRUE),cancelled=sum(is.na(arr_delay)),
n=n())|>filter(n>10)
```

Solutions (Style) — 1

```
flights |>
  filter(dest == "IAH") |>
  group_by(year, month, day) |>
  summarize(n = n(),
            delay = mean(arr_delay, na.rm = TRUE)) |>
  filter(n > 10)
```

Solutions (Style) — 2

```
flights |>
  filter(
    carrier == "UA",
    dest %in% c("IAH", "HOU"),
    sched_dep_time > 0900,
    sched_arr_time < 2000
  ) |>
  group_by(flight) |>
  summarize(
    delay = mean(arr_delay, na.rm = TRUE),
    cancelled = sum(is.na(arr_delay)),
    n = n()
  ) |>
  filter(n > 10)
```

Workflow: Scripts & Help

Scripts vs. Console

- **Console:** experiment, try things
- **Scripts** (`.R` files): save your work, reproduce later
- Open new script: **Ctrl + Shift + N**
- Run current line: **Ctrl + Enter**
- Run entire script: **Ctrl + Shift + S**

Use sectioning comments to organize:

```
# Load data ----
# Clean data ----
# Plot data ----
```

RStudio Projects

- A project = a folder + a `.Rproj` file
- Sets the **working directory** automatically
- All paths become **relative** to the project root
- **Never use** `setwd()` or absolute paths
- Create via: File → New Project

Getting Help

1. `?function_name` or `help(function_name)`
2. **Google** the error message
3. **Stack Overflow** and **Posit Community**
4. Create a **reprex** (reproducible example) with the `reprex` package
5. Include: minimal data, minimal code, the exact error